

Pan del Alba

Panadería web con Flask, Jinja2, SQLite y un volumen de Docker.

Objetivo

Construir una página de panadería que muestre productos y permita registrar pedidos. Los pedidos se guardan en SQLite dentro de un volumen para conservarlos al recrear el contenedor.

Piezas del proyecto

Componente	Función
Flask	Atiende la página y procesa el formulario.
Jinja2 + CSS	Muestra productos y pedidos con un diseño adaptable.
SQLite	Guarda cliente, producto, cantidad, total y fecha.
Dockerfile	Construye la imagen de Python con la aplicación.
Docker Compose	Publica el puerto 5000 y conecta el volumen pedidos_data.

Flujo de datos

Navegador → Flask → SQLite → volumen Docker. Al reiniciar el contenedor, Flask vuelve a leer el mismo archivo **panaderia.db** del volumen.

Resultado de la práctica

La aplicación se ejecutó en Docker y el pedido de prueba siguió visible tras eliminar y recrear el contenedor. Las capturas reales aparecen al final de este documento.

Docker paso a paso

Todos los comandos se ejecutan en la carpeta panaderia, con Docker Desktop iniciado.

Paso 1. Preparar la aplicación

Crear app.py, requirements.txt, templates/index.html y static/style.css. Flask escucha en el puerto 5000 y usa DATABASE_PATH para localizar la base SQLite.

Comprobación: Los cuatro archivos están en la carpeta del proyecto.

Paso 2. Escribir el Dockerfile

FROM selecciona Python 3.12; WORKDIR fija /app; COPY y RUN agregan los archivos e instalan Flask; ENV fija la ruta /app/data/panaderia.db; EXPOSE documenta el puerto y CMD inicia la aplicación.

Comprobación: Dockerfile existe junto a requirements.txt.

Paso 3. Configurar Compose

En docker-compose.yml, build: . usa el Dockerfile; ports publica 5000:5000; el volumen pedidos_data se monta en /app/data, donde SQLite guardará panaderia.db.

```
docker compose config
```

Comprobación: La configuración muestra web, puerto 5000 y volumen pedidos_data.

Paso 4. Construir la imagen

El primer comando lee el Dockerfile y produce la imagen local panaderia-web. Aún no se ha creado el contenedor.

```
docker compose build
```

```
docker image ls panaderia-web
```

Comprobación: panaderia-web aparece en la lista de imágenes.

Ejecutar y comprobar

Paso 5. Crear el contenedor

Compose inicia web-1 y crea la red y el volumen si no existen. El contenedor ejecuta Flask usando la imagen construida.

```
docker compose up -d
docker compose ps
docker compose logs web
```

Comprobación: El estado es running y se publica el puerto 5000.

Paso 6. Registrar un pedido

Abrir <http://localhost:5000>, escoger un producto y enviar el formulario. El pedido aparece en Pedidos recientes y se guarda en SQLite.

Comprobación: La prueba de tres marraquetas muestra un total de Bs 6.

Paso 7. Recrear el contenedor

down elimina el contenedor y la red; up crea un contenedor nuevo que vuelve a montar el mismo volumen. Al recargar la web, el pedido sigue visible.

```
docker compose down
docker compose up -d
docker compose ps
```

Comprobación: El ID cambió de 4481c631a587 a 53ee6d7e382a.

Paso 8. Comprobar el volumen

El volumen panaderia_pedidos_data conserva el archivo SQLite. down --volumes borraría ese volumen, por eso no se usa en esta prueba.

```
docker volume ls
docker volume inspect panaderia_pedidos_data
```

Comprobación: El volumen existe después de recrear el contenedor.

Planificación y aplicación

Estos prompts están preparados para guiar el trabajo. Si tu profesor pide evidencia de uso de IA, incluye capturas de los mensajes que realmente envíes.

1. Idea y estructura

«Quiero hacer una práctica de Docker con una página web sencilla para una panadería. Propón una estructura de proyecto con Flask, plantillas Jinja2, CSS, SQLite, Dockerfile y Docker Compose. La web debe mostrar productos y permitir guardar pedidos.»

Resultado: Estructura de carpetas y función de cada archivo.

2. Diseño visual con Jinja2

«Diseña una página para una panadería llamada Pan del Alba usando HTML, CSS y plantillas Jinja2. Debe mostrar productos con precios, un formulario para registrar pedidos y una lista de pedidos guardados. Haz que se adapte a celular.»

Resultado: templates/index.html y static/style.css.

3. Flask y SQLite

«Crea una aplicación Flask que muestre productos, reciba un formulario de pedido y guarde nombre del cliente, producto, cantidad, total y fecha en SQLite. Valida los datos del formulario y muestra los últimos pedidos en la página.»

Resultado: app.py, tabla pedidos y formulario funcional.

Docker y comprobación

4. Dockerfile

«Escribe un Dockerfile para una aplicación Flask de Python. Debe instalar las dependencias de requirements.txt, copiar la aplicación, exponer el puerto 5000 y arrancar el servidor. La base SQLite estará en /app/data/panaderia.db.»

Resultado: Imagen de la aplicación construida a partir del Dockerfile.

5. Compose y volumen

«Escribe un docker-compose.yml para la web Flask. Publica el puerto 5000 y monta un volumen con nombre en /app/data, de modo que panaderia.db permanezca aunque se elimine y recree el contenedor.»

Resultado: Servicio web y volumen pedidos_data.

6. Prueba de persistencia

«Explica cómo probar la página, registrar un pedido y demostrar que sigue guardado después de ejecutar docker compose down y docker compose up -d. Incluye los comandos para ver los contenedores, registros y volúmenes.»

Resultado: Procedimiento reproducible para verificar el volumen.

Comandos principales

docker compose build

docker compose up -d

docker compose ps

docker compose down

docker compose up -d

Para conservar los datos, evita docker compose down --volumes durante la prueba.

Evidencias y Git

Prueba del volumen

Se registró el pedido de prueba de tres marraquetas. El contenedor 4481c631a587 se eliminó con `docker compose down`. Docker Compose creó el nuevo contenedor 53ee6d7e382a y, al recargar la web, el pedido seguía visible porque la base de datos permaneció en el volumen `panaderia_pedidos_data`.

Capturas realizadas

1. Página web con catálogo, formulario y pedido después de recrear el contenedor.
2. Docker Desktop con el nuevo contenedor en ejecución y el puerto 5000.
3. Docker Desktop con el volumen `panaderia_pedidos_data`.

Repositorio y commits

github.com/Chucharizard/pan-del-alba-docker

1. feat: crear web de panaderia con Flask y SQLite
2. build: contenerizar la app y persistir pedidos en volumen
3. docs: explicar arquitectura, prompts y prueba de volumen
4. docs: añadir evidencias reales y PDF de entrega
5. docs: documentar creacion de imagen y contenedor paso a paso

Conclusión

La imagen contiene la aplicación, el contenedor la ejecuta y el volumen conserva los pedidos. La prueba se verificó con un pedido real de ejemplo y capturas de la ejecución.

Web y pedido guardado

Captura de la página después de recrear el contenedor. En Pedidos recientes sigue apareciendo el pedido de prueba de tres marraquetas.

Pan del Alba

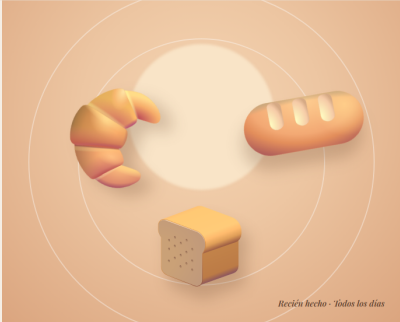
ProductosHacer pedidoPedidos

HORNEADO CON CARÍO, CADA MAÑANA

El sabor de volver a casa.

Pan crujiente, dulces recién hechos y ese aroma que hace que cualquier día empiece mejor.

Explorar productos



Recién hecho - Todos los días

HUESTROS FAVORITOS

Algo rico para cada momento.

Preparados con ingredientes sencillos y mucho cuidado.



Marraqueta **Bs 2**

Crujiente por fuera, suave por dentro.



Croissant **Bs 8**

Hojaldre dorado para acompañar tu café.



Pan de chocolate **Bs 10**

Una pausa dulce recién salida del horno.



Galletas caseras **Bs 6**

Hechas en casa, perfectas para compartir.

TU PEDIDO

Elige tu favorito. Nosotros lo preparamos.

Esta es una demostración para la práctica. Al enviar el formulario, tu pedido se guarda en SQLite.

Los pedidos permanecen guardados gracias al volumen de Docker.

Hacer un pedido

Tu nombre

Producto

Selecciona un producto

Cantidad

Guardar pedido

DATOS GUARDADOS

Pedidos recientes

Estos registros se leen desde la base de datos SQLite.

Pedido de prueba
Marraqueta - 3 unidades

Bs 6
25/09/2026 11:50

Pan del Alba

Hecho para la práctica de Docker + Flask + SQLite.

PAN DEL ALBA · PRÁCTICA 2

7

Contenedor y volumen

Docker Desktop muestra el contenedor web-1 con el puerto 5000 y el volumen panaderia_pedidos_data, que almacena panaderia.db.

